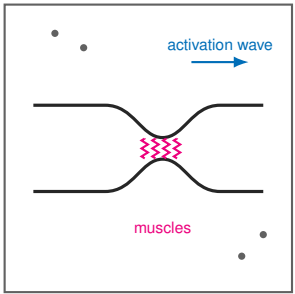
THE MODEL GALLERY, AND MUSCLE MODELS

Every fibre model IB2d ships and the example that shows it, the Python ones run and measured; then muscles: length–tension and force–velocity, the Hill models, crossbridges, calcium activation, and what IB2d's muscle code does with them



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4e

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Name every fibre model IB2d ships (springs, damped springs, torsional and non-invariant beams, target points, massive points, porous and poroelastic points, two muscle models, a user-defined law, tracers, concentration, Boussinesq, external forcing), its file and flag, and an example that shows it.
- Read what each model does from the runs: the shape of a beam relaxing, cylinders of different mass racing under gravity, a porous band collapsing, a scalar diffusing, a heart tube pumping with muscles, and check the numbers against a conservation law where there is one.
- Explain the muscle laws behind IB2d's muscle code: length–tension, Hill's force–velocity law, the combined model, the three-element Hill model; derive the force–velocity curve from crossbridge kinetics; describe how calcium activates a muscle and how twitches sum into tetanus.
- Write an activation function for IB2d's muscles, and know what the code does that the textbook model does not.

> Nine Python examples and seven variants ran on a desktop PC in October 2026 (WSL, Python IB2d, clone of 2024-03-06; 3–32 s each, the two big ones 21 and 29 minutes for a fraction of their shipped end times); the muscle laws are plotted and checked by three new scripts in `examples/IB2d/Muscle_Models`. Every measured number on these slides comes from those runs and scripts; the parameters are read from the examples' files and the biology from the lectures.

WHERE THIS CHAPTER COMES FROM

- "Fluid Structure Interaction (FSI) and the Immersed Boundary Method (IB)", N. A. Battista and L. A. Miller, 7 September 2016 (80 pages): when to use IB2d or IBAMR, the fibre models with their energies and forces, and the gallery (rubber band, wobbly beam, pulsing heart, mass and gravity races, artificial velocity, bifurcating artery, porous bands, the muscle heart tube). Its IBAMR part (pages 69–75) is chapters 2–3 and Part III.
- "Quick introduction to muscle models" (L. A. Miller, 15 September 2015, 38 slides): the sarcomere, length–tension and force–velocity, the combined model, the three-element Hill model, and the crossbridge derivation of Hill's law.
- "Muscle Activation Tutorial: the biology and Ca^{2+} models" (L. A. Miller, 29 slides): the motor neuron to the crossbridge, force summation, fibre types, tetanus, a calcium model for lamprey muscle (Williams et al.), activation-dependent stiffness.
- The IB2d code (clone of 2024-03-06): `EXAMPLE_LIST.txt`, the Python and MATLAB examples, the force routines (`please_Find_Lagrangian_Forces_On_Eulerian_grid`), the driver, the README.

What is new here: nine Python examples and seven variants run and measured (with a momentum check, a conservation check, a stability test of the porous bands); the 2016 talk's pictures complemented by the runs where the talk shows only the first frame; the muscle laws plotted and the crossbridge derivation checked numerically; what the muscle code does that the slides do not say; the textbook figures of the lectures (sarcomere, synapse) redrawn or described rather than copied (so are the lecture's crossbridge drawings, its half-sarcomere diagram and the ATPase–crossbridge cycle, slides 39 and 40).

IB2d or IBAMR?

IB2d:

- to test new fibre models or a computational model very easily;
- run codes locally;
- prepare simulations for IBAMR;
- get preliminary data;
- implement new fibre models; implement new fluid solvers.

IBAMR:

- run high-resolution simulations, e.g., greater than 256×256;
- speed;
- 3D applications;
- for publishable results.

> Still true in 2026. IB2d is a uniform-grid, periodic, single-process code: everything in this chapter runs in seconds at 32^2–128^2, while the two examples at 128×256 and 1024×128 take 2–3 hours each to their shipped end times on the desktop PC used here. IBAMR (Part III) adds adaptive refinement, MPI, 3D and walls without target points. Prototype here, then port (chapter 6 has the file-format differences).

THE FIBRE MODELS: FILE, FLAG, EXAMPLE (1/2)

model	file	`input2d` flag	Python example (chapter)
springs	`.spring`	`springs`	`Rubberband_with_Springs` (4a, 4d)
damped springs	`.d_spring`	`damped_springs`	`Rubberband_with_Damped_Springs` (4a)
torsional springs (“beams”)	`.beam`	`beams`	`Wobbly_Beam`, `Rubberband_with_Beams` (4a, 4b)
non-invariant beams	`.nonInv_beam`	`nonInvariant_beams`	`Wobbly_NonInv_Beam` (slide 10)
target points	`.target`	`target_pts`	`Moving_Rubberband`, `Pulsing_Heart` (4c, this repository)
massive points	`.mass`	`mass_pts`, `gravity`	`Gravity_Cellular_Race`, `Thrips` (slides 13–15)
porous points	`.porous`	`porous_media`	`Single_Porous_Rubberband` (4b, slide 20)
poroelastic points	`.poroelastic`	`poroelastic`	`Poroelastic_Rubberband` (slide 20)

Springs, damped springs, both kinds of beam and target points have an `update_*` partner (`update_springs`, `update_damp_springs`, `update_beams`, `update_nonInv_beams`, `update_target`) through which they can change in time (chapter 4c); masses, porous and poroelastic points have none. The file formats are in chapter 4a.

THE FIBRE MODELS: FILE, FLAG, EXAMPLE (2/2)

model	file	`input2d` flag	example
force–velocity / length–tension muscles	`.muscle`	`FV_LT_muscle`	`HeartTube_Muscle` (slides 33–35)
three-element Hill muscles	`.3_muscle` (MATLAB), `.muscle` (Python)	`3_element_muscle`	MATLAB `Example_Scallop` (slides 37–38); broken in Python
user-defined force	`.user_force`	`user_force_model`	`User_Defined_Fiber_Model` (slide 25)
tracers	`.tracer`	`tracers`	`Tracers_In_Impedance_Pump` (4b)
concentration	`.concentration`	`concentration`	`Concentration_Diffusion` (slide 22)
Boussinesq buoyancy	`boussinesq.mesh` (a fixed name, read by the example's own file)	`boussinesq`, `expansion_coeff` (with `gravity`, `concentration`)	`Rayleigh_Taylor_Instability` (slide 23)
external force on the grid	`please_Compute_External_Forcing.py`	`arb_ext_force`	`Turek_Hron` (slide 18)
electrophysiology, Brinkman, coagulation	various	`electro_phys`, `brinkman`, …	MATLAB only (slide 26; the Python driver has the Brinkman term, but its reader returns an undefined name)

The 2016 talk's list of fibre models: springs (Hookean or not), beams, torsional springs, target points, porosity, mass (with or without gravity), electromagnetic-type contact models, and muscle–fluid–structure models (the hybrid force–velocity / length–tension model and the three-element Hill model). The contact models are not in IB2d's example list.

THE PYTHON GALLERY: WHAT RAN, AND WHAT IT SHOWS

example	grid	steps	wall	what it shows
`Thrips`	32^2	1000	3.3 s	one massive point under gravity
`Poroelastic_Rubberband`	32^2	1000	3.8 s	a porous band whose points also drift with their spring force
`User_Defined_Fiber_Model`	32^2	1500	3.9 s	a spring law written by the user, identical to the built-in one
`Wobbly_NonInv_Beam`	32^2	5000	7.3 s	a curved beam with a straight rest shape
`HeartTube_Muscle`	128^2	500	9.5 s	a tube pinched by a wave of muscle activation
`Gravity_Cellular_Race`	64^2	7000	30 s	three cylinders of different mass falling
`Concentration_Diffusion`	64^2	5000	32 s	a scalar diffusing in a box of target points
`Rayleigh_Taylor_Instability`	128×256	40 000	29 min	heavy fluid over light: buoyancy from a concentration
`Turek_Hron`	1024×128	20 000	21 min	a flag behind a cylinder in driven channel flow
porous-band variants	32^2	to over 14 000	3–23 s	seven runs: with and without porosity, three time steps, one permeability

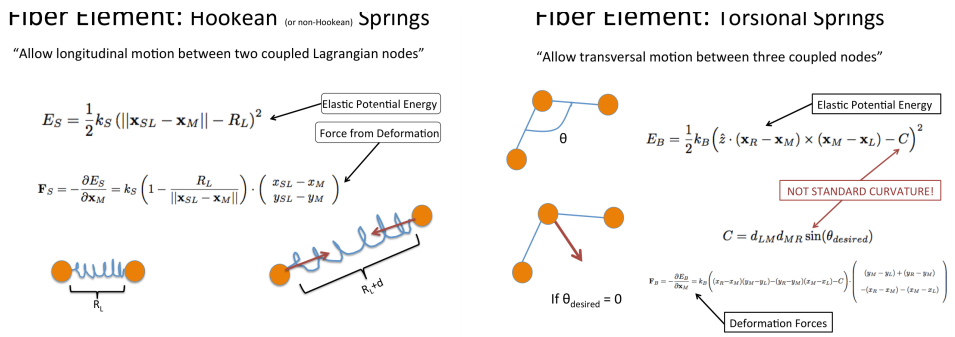
Run as shipped except: MATLAB plotting off, and T shortened for the two big ones (to 0.5 of 2 and 1 of 8; at the measured rates the shipped end times would take about 1.9 and 2.8 hours). Several ran three or four at a time, so the wall times are upper bounds. `wsl/final/ch4e_*.sh`; the copies under `/root/work/ch4/pyIB2d/Examples/*_4e` and `PorousCmp_*`.

THE MATLAB-ONLY EXAMPLES

matIB2d/Examples/	what it is (from the folder, its `input2d` and `EXAMPLE_LIST.txt`)
`Example_HeartTube`	the example list's seven heart tubes: FV–LT muscles, dynamic suction and peristaltic pumping by changing spring rest lengths, three-element Hill muscles, FitzHugh–Nagumo electrophysiology without and with calcium dynamics and with muscles (an eighth folder, named `128x128` but set to 128×64 on a 5×2.5 box); plus `Insect_Heart` (1024×256) and `Sea_Spiders` (1024×128)
`Example_Channel_Flow`	flow made by an external force: around a cylinder (two and one walls), a hole in the wall, flow in the middle, an empty channel, an aneurysm, Turek–Hron, a Brinkman channel, a poroelastic channel, concentration gradients (in a bifurcating artery; its impedance-pumping tube and pure-diffusion box have no forcing)
`Example_Mass_Points`	a massive cell in channel flow; cells racing in flow; racing under gravity; gravity against flow (64^2 and 256^2)
`Example_Tracers`	tracers in channel flow, an impedance pump, a peristaltic pump, a bifurcating artery
`Example_Porous_Rubberbands`, `Example_Poroelastic_Rubberband`	a single porous band; porous and non-porous side by side (64^2 and 256^2); poroelastic
`Example_Concentration_Dynamics`, `Example_Boussinesq`	concentration around corals, a leaf, moving cylinders and bands (constant, limited, sink); a falling sphere, Kelvin–Helmholtz and Rayleigh–Taylor instabilities
`Example_Jellyfish_Swimming`, `Example_Scallop`, `Example_Insect_Flight`	bells driven by resting lengths or interpolated phases ("not entirely debugged"); scallops with three-element or FV–LT muscles; a wing at 1024^2
`Example_Lymphangion`, `Example_VIV_Cylinder`, `Example_Coagulation`, `Example_Corals`	a pumping lymphatic vessel; vortex-induced vibration of a tethered cylinder (128^2–512^2); a coagulation model in a channel (stirring, through boundaries); corals whose target points move in time
`Example_KC`, `Example_Celestial_Mechanics`, `Example_Baby_Spider_Shower`, `Example_Skeleton`	target points that move between three shapes to spell a message ("Romance and Dating" in the 2016 talk); planets; baby spiders in a flow (springs, beams, mass, gravity, an external flow); a skeleton (template) for an example of your own
`Examples_First_Year_Seminar`, `Examples_Education`	12 short projects (beam and cells, beehive, corals, bubbles, jellyfish, leaf, plates, swimmer, ...); convergence, interpolation and a pendulum

Octave cannot run matIB2d (chapter 4a); these are described from their files, not run.

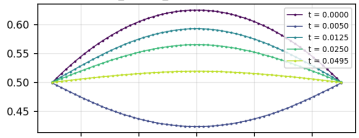
SPRINGS AND TORSIONAL SPRINGS: THE RECAP

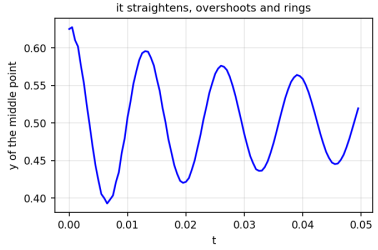


Springs allow longitudinal motion between two coupled nodes; torsional springs allow transversal motion between three. The talk's warning on the right: $C = d_{LM}\, d_{MR} \sin \theta_{\text{desired}}$ is not standard curvature but a cross product of the two segments, so it depends on the spacing. Chapter 4a has both force laws as coded (springs $\frac{1}{2}(\alpha+1)k(|d|-L)^{\alpha}$, torsional springs from the cross product) and runs the rubber band and the wobbly beam; chapter 4d measures the band's stiffness scaling.

The 2016 talk, pages 23 and 28 (Battista and Miller).

NON-INVARIANT BEAMS: WOBBLY_NONINV_BEAM





64 points from $(0.25, 0.5)$ to $(0.75, 0.5)$, starting as an arc (middle at $y = 0.625$); 62 non-invariant beams with $k_b = 10^{10}$ and a straight rest shape ($C_x = C_y = 0$); the two end points held by target points ($k_t = 2 \cdot 10^8$); $\mu = 0.1$, 32^2, $\Delta t = 10^{-5}$, $T = 0.05$: 5000 steps, 7.3 s.

The beam snaps through the straight line to $y = 0.39$ at $t \approx 0.0065$, back to 0.60, and rings with a period of about 0.013 and a slowly shrinking amplitude (the swing narrows from 0.39–0.60 to 0.45–0.56; at the last dump, $t = 0.0495$, the middle is at 0.52, rising). “Non-invariant” because the rest shape is a fixed pair (C_x, C_y) in the lab frame: rotate a beam with a curved rest shape and it no longer is at rest (chapter 4a, `.nonInv_beam`); a straight rest shape like this one ($C_x = C_y = 0$, evenly spaced points) is at rest in any orientation.

TARGET POINTS: THE GALLERY

A target point ties a Lagrangian node $\mathbf{x}_M$ to a virtual point $\mathbf{x}_M^T$ by a spring of zero resting length:

$$E_T(\mathbf{x}_M) = \tfrac{1}{2} k_T \, \|\mathbf{x}_M - \mathbf{x}_M^T\|^2, \qquad \mathbf{F}_T = -\frac{\partial E_T}{\partial \mathbf{x}_M} = -k_T \, (\mathbf{x}_M - \mathbf{x}_M^T).$$

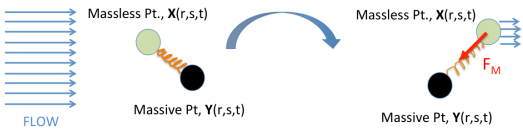
"Prescribe motion or make structure rigid": hold the virtual points still (walls, a cylinder) or move them (chapter 4c's update function).

- Pulsing heart (chapter 4c, ported and run): only target points, interpolated between two positions with a polynomial "to avoid instantaneous accelerations"; the talk colours it by pressure.
- Romance and dating (`Example_KC`): only target points, interpolated between *three* positions to spell a message; coloured by vorticity.
- Moving rubber band, sawtooth (chapter 4c); corals, insect flight at 1024^2, and every wall and cylinder in this chapter's channel examples.

The stiffness k_T trades rigidity against the time step (chapter 4d: on the rubber band a stiffer spring needs a smaller step; chapter 4c: on the sawtooth pump stiffer tethers failed sooner at the same step).

MASSIVE POINTS

"Gives the structure artificial mass using a *target-pt-spring-like* approach"



$$F_M = k_M\left(\mathbf{Y}_M - \mathbf{X}_M\right) \quad k_M >> 1$$ ← Hookean Restoring Force:

$$M(r,s)\frac{\partial^2 \mathbf{Y}}{\partial t^2} = -F_M - M(r,s)g\hat{e}_3$$ ← Move massive point position:

The 2016 talk, page 46.

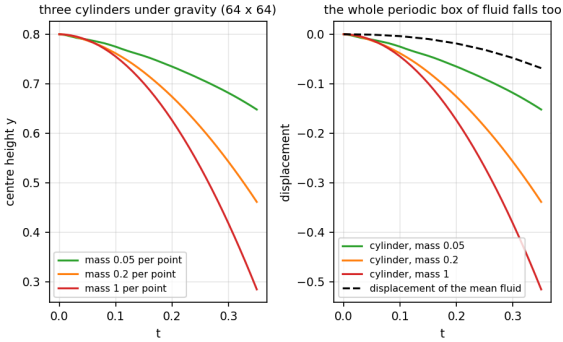
"Gives the structure artificial mass using a target-pt-spring-like approach." Each massless point $\mathbf{X}$ (which moves with the fluid and spreads force) has a massive twin $\mathbf{Y}$ that does not touch the fluid; a stiff spring ties them:

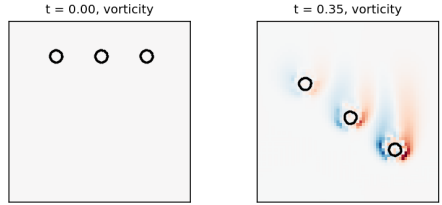
$$\mathbf{F}_M = k_M\left(\mathbf{Y} - \mathbf{X}\right) \text{ on } \mathbf{X}, \qquad M\,\ddot{\mathbf{Y}} = -\mathbf{F}_M + Mg\,\hat{\mathbf{g}}.$$

In the code (`give_Me_Mass_Lagrangian_Force_Densities`, `please_Update_Massive_Boundary_Velocity`):

- `.mass` rows are `index k_M M`; $\mathbf{Y}$ starts on $\mathbf{X}$;
- $g = 9.80665$ is hard-coded; the direction $\hat{\mathbf{g}}$ is the normalised (`x_gravity_vec_comp`, `y_gravity_vec_comp`);
- M is a mass per unit length, like every IB2d force a density: the fluid feels $k_M(\mathbf{Y} - \mathbf{X})\,\Delta s$ with the driver's $\Delta s = L_x/(2N_x)$ (chapter 4d), so a point's weight on the fluid is $Mg\,\Delta s$. The next two slides check this.

THE GRAVITY RACE: GRAVITY_CELLULAR_RACE





Three cylinders of 54 points (radius $1/30$, centres $(0.25, 0.8)$, $(0.5, 0.8)$, $(0.75, 0.8)$), springs $k = 5 \cdot 10^7$ and beams $k_b = 10^7$ around each (and 27 springs across each, $k = 5 \cdot 10^5$, resting length the diameter), $k_M = 10^6$, and $M = 0.05$, 0.2 and 1 per point; gravity $(0, -1)$; $\mu = 0.01$, 64^2, $\Delta t = 5 \cdot 10^{-5}$, $T = 0.35$: 30 s. (The generator is called `Channel_Channel.py`, but there is no channel: only the three cylinders in the periodic box.)

At $t = 0.35$ the cylinders have fallen 0.152, 0.339 and 0.515, and are still accelerating (speeds 0.72, 1.70, 2.80). The heavier the cylinder, the stronger the wake (right). And the whole box of fluid falls too: its mean vertical velocity is -0.45 at $t = 0.35$ and growing (dashed line: the mean fluid's displacement).

WHY THE WHOLE FLUID FALLS: A MOMENTUM CHECK

t	fluid	masses	sum	weight $\times\, t$
0.05	-0.034	-0.227	-0.260	-0.259
0.10	-0.086	-0.433	-0.519	-0.517
0.15	-0.147	-0.627	-0.774	-0.776
0.20	-0.217	-0.818	-1.036	-1.034
0.25	-0.292	-1.002	-1.294	-1.293
0.30	-0.368	-1.183	-1.552	-1.551
0.35	-0.453	-1.339	-1.792	-1.810

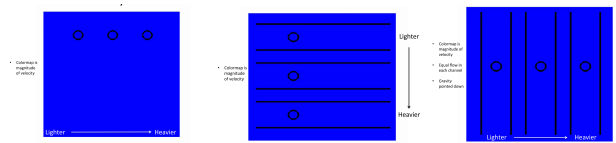
Vertical momentum: the fluid's ($\rho\times$ mean $u_y\times$ box area) plus the cylinders' ($\sum 54\, M\, \Delta s\times$ the centre's speed), against the total weight $\sum 54\, Mg\, \Delta s = 5.17$ times t.

The two columns agree to 0.5 % until $t = 0.30$ (1 % at 0.35, the last dump, where the speeds are one-sided differences): Newton's second law for fluid plus cylinders, with the weight $Mg\,\Delta s$ per point. That confirms the per-unit-length reading of M and $\Delta s = L_x/(2N_x)$.

- In a periodic box nothing holds the weight up: no walls, no bottom. The pressure of a periodic solver has no mean gradient, so the whole fluid accelerates with the cylinders. "Terminal velocity" never comes.
- For a settling problem, put walls in (target points: the MATLAB `Gravity_vs_Flow` and `Channel_Flow_Cellular_Race` run in channels, but the MATLAB `Gravity_Cellular_Race`, like the Python one, has none), or subtract the mean force from the fluid each step (the option `balance_periodic_force` of the `python-updates-2026` branch, pull request 34 to IB2d) and measure speeds relative to the fluid.

THRIPS, AND THE MATLAB RACES

`Thrips`: one massive point at $(0.25, 0.4113)$ in a 0.5×0.5 box, gravity on, $\mu = 0.1$, 32^2, $T = 0.1$. It falls $1.1 \cdot 10^{-5}$ in 0.1 time units (speed $1.7 \cdot 10^{-4}$ at the end), where free fall would be 0.049: its weight on the fluid is $Mg\,\Delta s = 0.001 \times 9.807 \times 0.0078 = 7.7 \cdot 10^{-5}$. The folder is a stub: `make2d.py` would write $k_M = 10^6$, $M = 6 \cdot 10^{-8}$, but the shipped `thrips.mass` holds $k_M = 1000$, $M = 0.001$ (and a second geometry, `thrips2.*`, is unused).

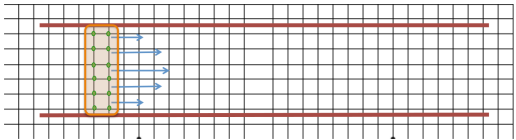


The MATLAB races of the 2016 talk (set-ups, first frames): “Mass vs. gravity cell race”, “Channel flow mass race”, “Mass–gravity–flow race” (equal flow in each channel, gravity down); colour is the speed. `Example_Mass_Points/Gravity_Cellular_Race`, `Channel_Flow_Cellular_Race`, `Gravity_vs_Flow`.

The 2016 talk, pages 47, 48 and 54.

ARTIFICIAL VELOCITY: AN EXTERNAL FORCE ON THE GRID

"Add artificial force to Eulerian grid to get desired background velocity profiles"



$$F_{ext} = -k_{ext}\Big(\mathbf{u}(\mathbf{x},t) - \mathbf{u}_{aux}(\mathbf{x},t)\Big)$$ ← Artificial Force to Eulerian Grid

$$\mathbf{u}_{aux} = \begin{pmatrix} U_{max}\left(1 - \left(\frac{y}{d/2}\right)^2\right) \\ 0 \end{pmatrix}$$ ← Desired Velocity Profile

The 2016 talk, page 52.

"Add artificial force to Eulerian grid to get desired background velocity profiles": in a box of grid cells, a penalty force pulls the fluid towards a target velocity,

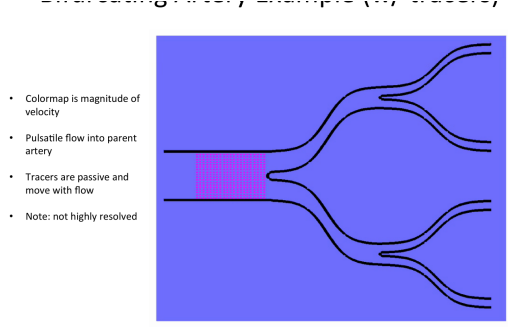
$$\mathbf{F}_{\text{ext}} = -k_{\text{ext}}\left(\mathbf{u}(\mathbf{x},t) - \mathbf{u}_{\text{aux}}(\mathbf{x},t)\right),$$

with a parabolic $\mathbf{u}_{\text{aux}}$ across the channel. The example supplies `please_Compute_External_Forcing.py` and sets `arb_ext_force = 1`; in `Turek_Hron` the box is $0.1 \le x \le 0.16$, $0.035 \le y \le 0.215$ and

```
uX_Tar = -Umax * (100 * np.tanh(2*t)) * ( (Ly/2+w/2) - y ) * (
      (Ly/2-w/2) - y )
fx[j,i] = fx[j,i] - kStiff*( uX[j,i] - uX_Tar )
```

($U_{\max} = 2.75$, $w = 0.2$, $L_y = 0.25$, $k = 10^3$): a parabola that vanishes at the walls $y = 0.025$ and 0.225, peaks at $2.75 \tanh 2t$ on the centre line, and ramps up over $t \sim 0.5$.

CHANNEL FLOWS, AND THE BIFURCATING ARTERY



The 2016 talk, page 53:

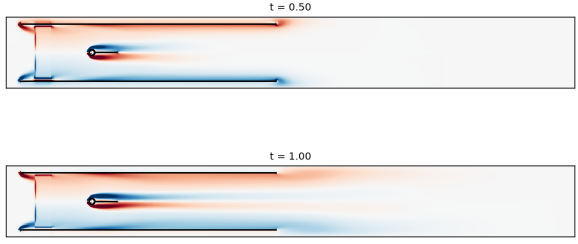
`Example_Tracers/Tracers_In_Bifurcating_Artery`.

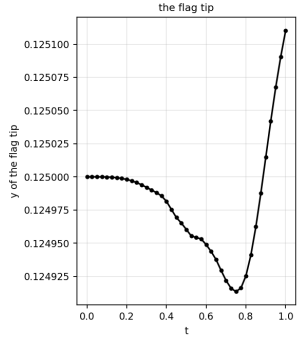
Bifurcating artery with tracers: pulsatile flow into the parent artery from an external force, passive tracers that move with the flow, colour the speed; “not highly resolved”.

`Example_Channel_Flow`: the same forcing makes every channel flow of IB2d: around a cylinder between two walls or next to one, through a hole in the wall, in the middle of the domain, in an empty channel, in an aneurysm, past a cylinder with a flag (Turek–Hron, next slide), through a Brinkman medium, past a poroelastic obstacle, carrying a concentration (the bifurcating artery of `Example_Concentration_Gradients`; its other two are not forced).

Three things to know: the walls are target points (there are no wall boundary conditions in a periodic code); what leaves the right edge comes back in at the left, so the forcing box also has to remove the wake; and the force is a penalty, so the velocity in the box only approaches the target, as closely as the stiffness k and the box’s size allow (the fluid is incompressible, so the box has to drive all the fluid between the walls: chapter 4f).

TUREK_HRON: A FLAG BEHIND A CYLINDER

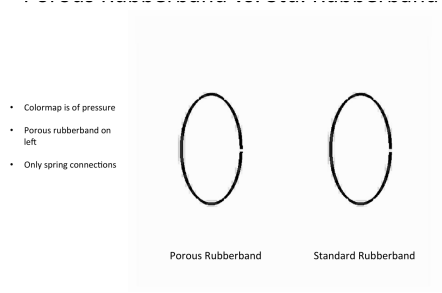




A 2×0.25 periodic box, 1024×128; a channel of target-point walls at $y = 0.025$ and 0.225 from $x = 0.05$ to 0.95 (the right half of the box is open fluid, where the wake leaves the channel); a cylinder of radius 0.01 at $(0.30, 0.125)$, held by target points; a flag of 80 points from $x = 0.313$ to 0.390 (springs, beams, mass $M = 5$ per point); the forcing box of slide 16 at the inlet. Walls and cylinder points carry mass as well as targets. $\mu = 7.5 \cdot 10^{-4}$, $\Delta t = 5 \cdot 10^{-5}$; run to $T = 1$ (shipped: 8): 20 000 steps, 21 minutes.

Vorticity at $t = 0.5$ and 1: shear layers on the walls and the cylinder, the forcing box (the rectangle at the inlet), the jet leaving the channel at $x = 0.95$. By $t = 1$ the flag tip has moved by 10^{-4} (right): the inflow has only just reached full speed ($2.75 \tanh 2t$), and the flapping the benchmark measures needs more time (the shipped run goes to $t = 8$). The proportions are not the benchmark's (Turek and Hron: channel $4.1D$ wide, flag $3.5D$ long; here $10D$ and $3.9D$), so compare shapes, not numbers.

POROUS POINTS



The 2016 talk, page 59: a porous band (left) and a standard band, coloured by pressure.

"Makes the structure permeable to fluid flowing through it." Darcy's law gives the slip of the fluid through the membrane relative to the boundary,

$$U_p\,\hat{\mathbf{n}} = -\frac{\kappa\,[p]}{\mu\,a} \qquad (\kappa \text{ permeability}, a \text{ thickness}, [p] \text{ pressure jump}),$$

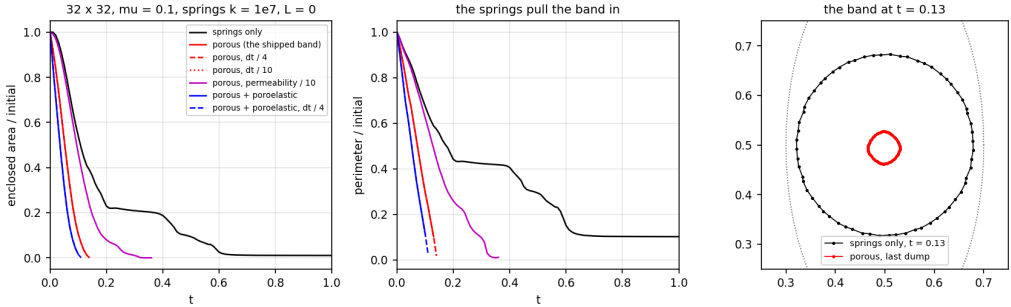
and the boundary moves with the fluid velocity plus the slip. In IB2d the pressure jump is the boundary's own force: `please_Compute_Porous_Slip_Velocity` sets

$$U_{p,x} = -\kappa_i\,F_x\,n_x/|\mathbf{X}_s|, \qquad U_{p,y} = -\kappa_i\,F_y\,n_y/|\mathbf{X}_s|,$$

with the normal from a five-point derivative along the band. `.porous` rows are `index kappa stencil`, the last column ($-2\ldots2$) saying where the point sits in the stencil (ends of open structures). The talk notes "only spring connections" in its picture.

Poroelastic points (`.poroelastic`: `index alpha`) add a second drift: each listed point moves by $\Delta t\,\mathbf{F}_{\text{spring}}/(\mu\,\alpha)$ every step (`Supp.py`, lines 116–122; the file calls α a "Brinkman constant").

THE POROUS BANDS, RUN PAST THEIR END TIME

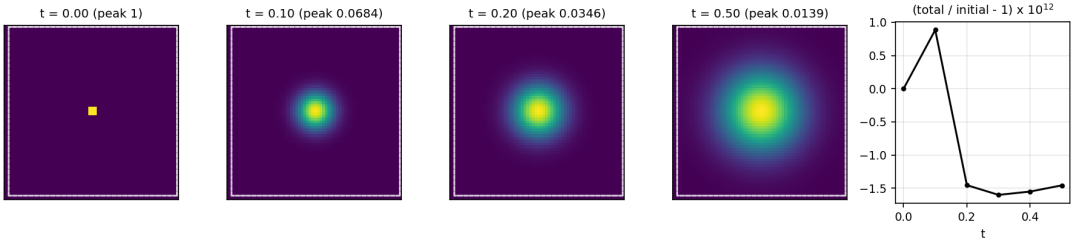


Both shipped bands are stiff: 64 points, springs $k = 10^7$, resting length 0, $\mu = 0.1$, 32^2, $\Delta t = 10^{-4}$, and they stop at $T = 0.1$. Run to $T = 1$: the band without porosity keeps 54 % of its area at $t = 0.1$, 23 % at 0.2, 2.5 % at 0.6 and 1 % from $t \approx 0.7$ on (the method's volume loss, chapter 4d, at a tension 400 times the standard band's); the porous band keeps 10 % at $t = 0.1$ and stops at $t = 0.13$; the poroelastic band stops at $t = 0.10$. Why they stop: next slide.

WHY THE POROUS BANDS STOP

- The stops are not a time-step problem: at $\Delta t/4$ and $\Delta t/10$ the porous band stops at $t = 0.14$, and at $\Delta t/4$ (the only smaller step tried for it) the poroelastic at 0.11; their area curves lie on those at Δt.
- With the permeability divided by ten it lasts to $t = 0.36$.
- Each time the band has collapsed by its last dump (area 0–1.5 % of the start, perimeter 1–11 %, closest neighbours $5 \cdot 10^{-5}$ to $3 \cdot 10^{-3}$ apart, a seven-hundredth to an eleventh of a cell): springs of zero resting length pull a porous band to a point, where the code divides by distances that vanish (the spring directions, and the porous slip's $|\mathbf{X}_s|$; which fails first was not traced), and the run ends with chapter 4d's `IndexError` (all six logs).
- The sealed band is at area 1 %, perimeter 10 % and neighbours $5 \cdot 10^{-4}$ apart by $t = 1$, and runs on. The shipped $T = 0.1$ ends just before.

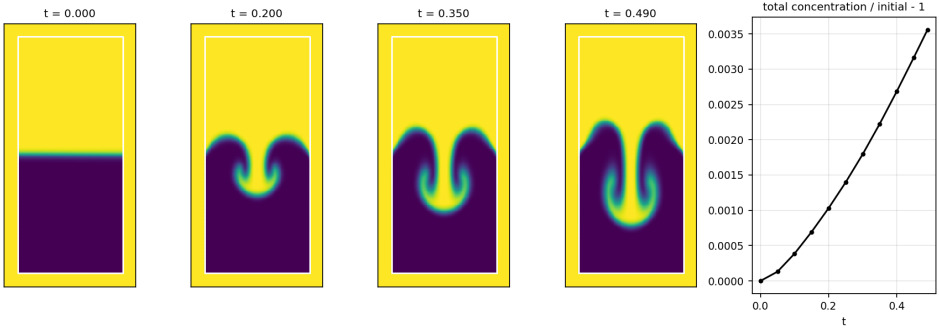
A CONCENTRATION: CONCENTRATION_DIFFUSION



`concentration = 1` reads `<name>.concentration`: its first line is the diffusion coefficient (here 0.025), then the initial field on the grid (N_y rows of N_x values). The driver advects and diffuses it with the flow every step. Here: a 3×3-cell square of $C = 1$ in the middle of a box walled by target points ($k = 10^7$), $\mu = 10$ (and no flow at all: $|\mathbf{u}| = 0$ in every saved velocity field), 64^2, $T = 0.5$: 32 s.

The peak falls from 1 to 0.068 at $t = 0.1$ and 0.014 at $t = 0.5$; the spread grows like $\sqrt{2Dt}$ (0.16 at $t = 0.5$). The total amount $\sum C\,\Delta x\,\Delta y = 0.002197$ is conserved to $2 \cdot 10^{-12}$ of itself (right): with the fluid at rest only the diffusion part acts, and it conserves; the cheapest check of a concentration run (slide 24: the upwind advection fails it once the fluid moves). A `source` flag and the `Example_Concentration_Dynamics` folders (corals, a leaf, a sink) add production and uptake in MATLAB.

BUOYANCY: RAYLEIGH_TAYLOR_INSTABILITY



A 0.5×1 box, 128×256, holding a tub of 2660 target points ($0.05 \le x \le 0.45$, $0.05 \le y \le 0.95$); a concentration $C = 1$ (heavy) above $y = 0.5$ in the tub and outside it, $C = 0$ (light) below. `boussinesq = 1` adds the force $\rho\,\beta\,g\,C\,\hat{\mathbf{g}}$ on the 23 793 grid cells listed in `boussinesq.mesh` (the inside of the tub), with $\beta =$ `expansion_coeff` $= 10$ and $g = 0.981$ written into the example's own `please_Form_Boussinesq_Forcing_Terms.py` (not the 9.80665 of the mass points). $\mu = 0.01$, $\Delta t = 1.25 \cdot 10^{-5}$; run to $T = 0.5$ (shipped: 2): 40 000 steps, 29 minutes.

BUOYANCY, CONTINUED: A SEEDED PLUME, AND A SCHEME THAT DOES NOT CONSERVE

The mushroom at $t = 0.49$ is seeded, not driven: the example also sets `arb_ext_force = 1`, and its `please_Compute_External_Forcing.py` pulls the fluid in the box $0.2 \le x \le 0.3$, $0.48 \le y \le 0.52$ (the middle of the interface) towards $u_y = -10 \tanh 20t$ with $k = 10^4$, but only while $t < 0.01$ (`if (t<0.01):`, the first 800 steps; the target has then reached only $-10 \tanh 0.2 \approx -2$): a short downward kick at the middle of the interface that seeds one plume, which buoyancy then grows. The total concentration is *not* conserved here: it rises by $0.36\,\%$ by $t = 0.49$ (right). The scheme is first-order upwind in advective form, $C \leftarrow C + \Delta t\,(k\,C_{xx} - u\,C_x)$, split in x and y (`Supp.py`, `perform_Time_Split_Upwind`), which conserves the total when the fluid is at rest (slide 22) but not in a flow that varies in space (a uniform flow would conserve it too, were it not for the loops below). And its Python loops stop one cell short: `range(1, lenX-2)` skips the column `lenX-2` (MATLAB's `for j=2:lenX-1` does not), likewise the row `lenY-2`, so nothing is advected there (in this run both lie outside the tub, at $x = 0.492$ and $y = 0.992$, where $C = 1$ stays uniform, so the skip is not what adds the 0.36 %).

> Two lessons for any example you take from a gallery: read every flag that is on and the function behind it (here `arb_ext_force`, whose forcing switches itself off at $t = 0.01$), and check the conserved quantities (here the total concentration) before you read the picture.

A FORCE LAW OF YOUR OWN: USER_DEFINED_FIBER_MODEL

```
def give_Me_General_User_Defined_Force_Densities(ds,Nb,xLag,yLag,
          xLag_P,yLag_P,dt,current_time,general_force):
    Nsprings = general_force.shape[1]  # # of Force Connections
    id_Master = general_force[0,:].astype('int')
    id_Slave = general_force[1,:].astype('int')
    K_Vec = general_force[2,:]  # spring stiffness
    RL_Vec = general_force[3,:] # spring resting length
    fx = np.zeros(Nb); fy = np.zeros(Nb)
    dx = xLag[id_Slave] - xLag[id_Master]
    dy = yLag[id_Slave] - yLag[id_Master]
    sF_x = K_Vec * (np.sqrt(dx**2 + dy**2)-RL_Vec) * (dx/np.sqrt(dx**2+
      dy**2))
    sF_y = K_Vec * (np.sqrt(dx**2 + dy**2)-RL_Vec) * (dy/np.sqrt(dx**2+
      dy**2))
    for ii in range(Nsprings):
        fx[id_Master[ii]] += sF_x[ii]; fy[id_Master[ii]] += sF_y[ii]
        fx[id_Slave[ii]] -= sF_x[ii];  fy[id_Slave[ii]] -= sF_y[ii]
    return (fx, fy)
```

The example's `give_Me_General_User_Defined_Force_Densities.py`, comments shortened. The driver reads `rubberband.user_force` (rows `i j k L`) transposed: one column per connection.

`user_force_model = 1`: the driver imports this function from the example's folder every step, passes the current and previous positions, Δt and the time, and adds the forces it returns to all the others. Anything you can compute from the points goes here: a contact force, a nonlinear membrane, an anisotropic law.

The example's law is a Hookean spring, $k = 2.5 \cdot 10^4$, $L = 0$, on the standard rubber band: the run (1500 steps, 3.9 s) is identical to the last digit to the springs run of chapter 4d at dumps 0, 10, 25, 50 and 75. That is how to test a new law: first reproduce a built-in one exactly, then change it.

WHAT ELSE SHIPS, BRIEFLY

- Damped springs (`.d_spring`: `i j k L b`): a spring plus a damping force on the leader's velocity (chapter 4a ran `Rubberband_with_Damped_Springs`); used to tether the cylinder in two of the seven `Example_VIV_Cylinder` folders (`Single_VIV_Cylinder_Damped_Springs_256` and `_High`, $b = 5$ and 50), and for all 153 springs of `Scallop_3Elem_Muscle` (along and across its shell, with $b = 0$: no damping).
- Electrophysiology (`electro_phys`, MATLAB): a FitzHugh–Nagumo excitable medium along a heart tube whose potential sets the stiffness of the tube's springs, $(16V)^4$ (`IBM_Driver.m`, line 800; `Example_HeartTube/Electromechanical_Pumping*`), or, with `FV_LT_muscle = 1`, drives the muscles (`EM_Pump_w_Calcium_and_Muscles`); in the folders with calcium, the stimulus is $0.5\,Caf(t)$ from a calcium model (`Calcium_Dynamics.m`).
- Brinkman (`brinkman`): a drag $-\mu\, B\, \mathbf{u}$ added to the fluid force where the field B (an inverse permeability, though the code calls it the permeability matrix; read from a file, written to `viz_IB2d`) is non-zero; `Example_Brinkman_Channel`.
- Coagulation (`coagulation`, MATLAB): a coagulation model in a channel, stirred or tested through boundaries (`Example_Coagulation`).
- Restart (`Restart_Flag`, MATLAB): `Example_Test_Restart_Protocol` restarts a standard and a poroelastic band from saved data.
- Tracers (chapter 4b): passive points that move with the flow and exert no force (`.tracer`); the Python tracer call needed a fix (pull request 34).

PART 2: MUSCLE MODELS

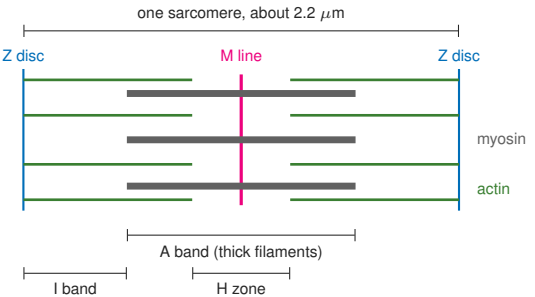
The 2015 lecture's outline, extended by the activation tutorial:

1. A simple muscle model that describes the force produced as a function of the muscle fibre's length and velocity (the model IB2d's `FV_LT_muscle` uses).
2. The three-element Hill model (contractile, series and parallel elements; IB2d's `3_element_muscle`).
3. A microscale model of crossbridge formation and detachment, from which Hill's force–velocity law follows.
4. Activation: from the motor neuron to calcium, force summation, and a calcium model that turns a stimulus train into the activation that multiplies the force.

Then IB2d's muscle code: what the heart tube does, what it computes that the models do not say, and what is broken.

> A muscle in IB2d is a force between two points, along the line joining them, whose size depends on their distance (length), how fast that distance changes (velocity), and an activation you write as a function of time and position. Everything below is about choosing those three.

THE SARCOMERE AND THE SLIDING FILAMENTS

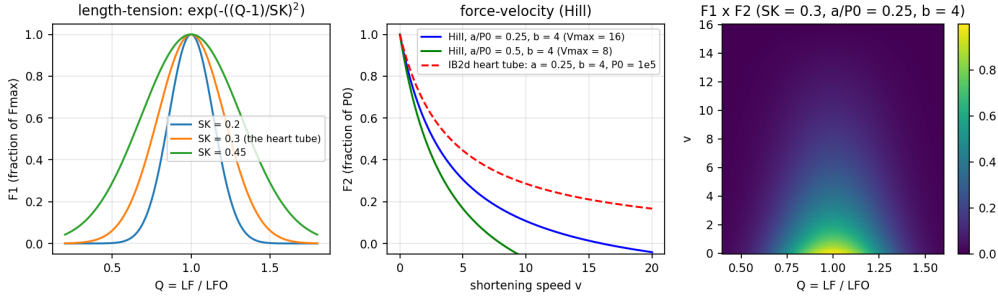


Thin (actin, green) filaments anchored at the Z discs overlap the thick (myosin, grey) filaments; contraction slides them past each other.

- Sarcomeres are the fundamental units of muscle contraction. Arranged in series, the muscle's total change in length may be large while each sarcomere's is small.
- Each is made of thick (myosin) and thin (actin) filaments.
- During contraction the dark bands keep their length, the light bands shorten, and the myofibril shortens: the filaments slide, they do not shorten.
- The force produced is a function of the speed of contraction and the length of the muscle.

After the 2015 lecture's slides 4–5 (which use figures 16-69 and 16-71 of Alberts et al., *Molecular Biology of the Cell*, 4th ed.; redrawn here).

LENGTH–TENSION AND FORCE–VELOCITY



Length–tension. The tension a sarcomere can generate depends on its length: at intermediate lengths all myosin heads are within reach of thin filaments; stretched, few overlap; fully contracted, the filaments interfere. (The lecture's figure, unattributed: maximal force for sarcomere lengths of about 1.8–2.2 μm, zero near 1.3 and 3.6 μm, a knee at about 1.6 μm and 80 %; Gordon, Huxley and Julian's frog fibres (1966): shoulder at 1.67 μm, zero at 3.65 μm.) The smooth model IB2d uses:

$$F_I = F_{IO} \exp\Big[- \Big(\frac{Q-1}{SK} \Big)^2 \Big], \qquad Q = \frac{L_F}{L_{FO}},$$

L_{FO} the length of maximal tension, SK a constant for each muscle (the lecture: $SK > 0.28$).

The 2015 lecture, slides 6–8; the 2016 talk, pages 62–64; `Muscle_Models/muscle_curves.py`.

FORCE–VELOCITY: HILL'S LAW

Force–velocity (Hill's law for a fully activated, tetanic contraction, a steady-state model):

$$(P + a)(V + b) = b\,(P_0 + a) \iff V = b\,\frac{P_0 - P}{P + a},$$

V the shortening speed, P_0 the maximal (isometric) load, $V_{\max} = bP_0/a$ the speed with no load; a and b are fitted for each muscle ($a/P_0 \approx 0.25$ in Hill's frog muscle). Normalised:

$$F_2(V) = \frac{P}{P_0} = \frac{b - (a/P_0)\,V}{V + b}.$$

The 2015 lecture, slides 6–8; the 2016 talk, pages 62–64; `Muscle_Models/muscle_curves.py`.

THE COMBINED MODEL, AND IB2D'S CODE FOR IT

Assume the force for a given length and velocity is the product of the normalised laws, scaled by an activation:

$$F_{\text{muscle}}(L_F, V_F) = a_F \, F_{\max} \, F_1(L_F) \, F_2(V_F),$$

$a_F \in [0, 1]$ the normalised degree of activation, $F_{\max}$ the maximal isometric force, F_1 the normalised length–tension law, F_2 the normalised Hill law.

"Active force comes from force generated by actin & myosin crossbridges at [sarcomere] level. It is fully extensible when inactive, but shortens when active!" (the 2016 talk; the 2015 lecture's slide 12 has the longer form it shortens)

```
# .muscle rows: leader follower LF0 SK a b Fmax
dx = xLag[id_Follower] - xLag[id_Leader]
dy = yLag[id_Follower] - yLag[id_Leader]
LF = np.sqrt( dx**2 + dy**2 )          # length now
LF_P = np.sqrt( dx_P**2 + dy_P**2 )    # length one step ago
v =  np.abs(LF-LF_P)/dt                # speed
from give_Muscle_Activation import give_Muscle_Activation
Fm = give_Muscle_Activation(v,LF,LF0_Vec,SK_Vec,a_Vec,b_Vec,
                            FMAX_Vec,current_time,xPt,np.array(xLag
      ))
mF_x = Fm*(dx/LF); mF_y = Fm*(dy/LF)
fx[id_Leader] += mF_x; fx[id_Follower] -= mF_x   # (in a loop)

# give_Muscle_Activation.py, in the example's folder:
Q = LF/LF0
F1 = np.exp(-((Q-1)/SK)**2)
F2 = (1/P0)*(b*P0-a*v)/(v+b)           # P0 = Fmax
Fm = af_Val*Fmax*F1*F2                 # af from the activation
      wave
```

`give_Muscle_Force_Densities` and the heart tube's activation file, abridged. A positive F_m pulls the two points together.

The 2015 lecture, slides 9–10; the 2016 talk, pages 60–65.

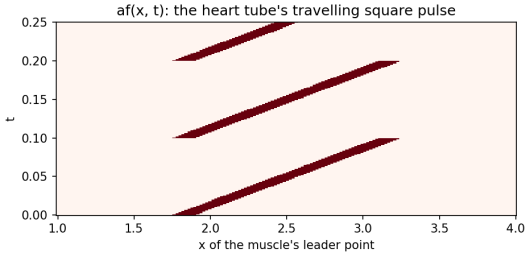
THREE THINGS THE CODE DOES THAT THE MODEL DOES NOT SAY

1. The speed has no sign. `v = |LF - LF_P|/dt`: a muscle being stretched is treated as one shortening at the same speed, so its force drops along Hill's curve. Real muscle resists lengthening with *more* than P_0 (eccentric contraction); Hill's law is for shortening only. Both ports do this (MATLAB: `v = abs(LF-LF_P)/dt`).
2. a is in force units. $F_2 = (bP_0 - av)/(P_0(v+b))$ with the heart tube's $a = 0.25$, $b = 4$, $P_0 = F_{\max} = 10^5$ is $4/(v+4)$ to within a relative $6 \cdot 10^{-7}\, v$ (six digits for $v \lesssim 1$, four to five at the run's fastest muscles, $v \approx 85$ between dumps): $V_{\max} = bP_0/a = 1.6 \cdot 10^6$, so the muscle never stalls. If 0.25 was meant as Hill's a/P_0, the file should hold $a = 2.5 \cdot 10^4$ (the red dashed curve against the blue one on slide 29).
3. The activation sees only the leader's x. `give_Muscle_Activation` is called with the leader point's x and the whole `xLag`; any dependence on y, on the follower, on the muscle's own state (fatigue, calcium) has to be built from those.

> None of these is a bug in the sense of a crash; they are modelling choices made in the code and not in the slides. Read the force routine before you trust a parameter's units.

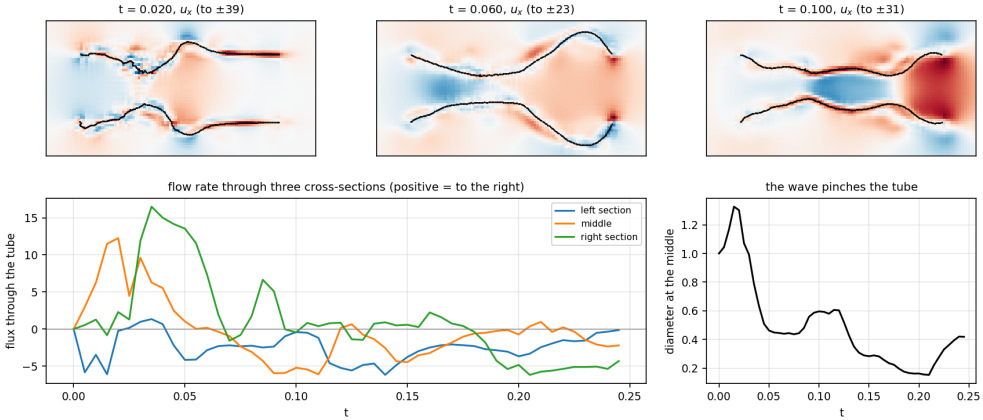
THE HEART TUBE WITH MUSCLES: `HEARTTUBE_MUSCLE`

- A tube of diameter $d = 1$ and length 3 (two rows of 155 points, x from 1 to 4, $y = 2$ and 3) in a 5×5 box, 128^2, $\mu = 0.1$, $\Delta t = 5 \cdot 10^{-4}$, $T = 0.25$.
- Springs $k = 10^7$ along each wall, beams $k_b = 7.5 \cdot 10^7$ (straight), one target point at each end of each wall ($k_t = 10^6$): the tube floats.
- 153 muscles across the tube, each from a point of one wall to the point opposite: $L_{FO} = d = 1$, $SK = 0.3$, $a = 0.25$, $b = 4$, $F_{\max} = 10^5$.
- The activation: a square pulse, width $L_{AR}/10 = 0.15$, that sweeps the middle half of the tube ($1.75 \le x \le 3.25$) left to right at speed 13.5, once every 0.1 (`freq = 10`), then starts again: 7 or 8 muscles on at a time (right).



`Muscle_Models/activation_wave.py`: the example's activation function copied line for line, evaluated on the example's points without the fluid (what its `test_Activation_Wave.py` does).

THE HEART TUBE, RUN



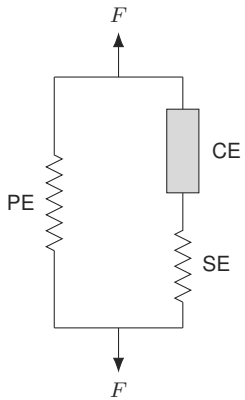
Top: u_x (red to the right, blue to the left) at $t = 0.02, 0.06, 0.10$, each frame on its own scale. Bottom left: the flow rate $\int u_x \, dy$ across the tube at $x = 1.75, 2.5$ and 3.25. Bottom right: the tube's diameter at $x = 2.5$. 500 steps, 9.5 s.

READING THE HEART TUBE

- The pinch is strong: the diameter at the middle ($x = 2.5$) first bulges to 1.33 ($t = 0.015$, before the first pulse arrives), falls to 0.46 as that pulse passes ($t \approx 0.05$) and keeps falling, to 0.15 at $t = 0.21$ (the third pulse is then still upstream), before it reopens to 0.42: the wall lags the activation. With only four target points the whole tube flexes and its ends flail. The first frames carry a start-up transient on the walls (chapter 4b saw the same).
- The flow oscillates with every pulse (rates from -6.2 to $+16.5$), and its mean over the last full period ($0.15 \le t < 0.25$) is negative at all three sections: -2.2, -1.3 and -2.9; over the first period ($0 \le t < 0.1$) it was -2.0, $+2.0$ and $+5.4$. In this run the tube moved fluid *to the left*, against the activation wave, only after its first period; which way it pumps once it is periodic is open. The three means differ by more than the noise: in 2.5 periods the tube's volume has not settled into a periodic state.
- The talk shows this example with the warning "PARAMETERS ARE NOT FINELY TUNED!!!!!". A pumping study starts here: longer runs (many periods), walls held better, the flux over whole periods at several sections until they agree, then the parameters (pulse speed, width, $F_{\max}$, the a of slide 32).

> Measure what a pump does over whole periods and at more than one section before you say which way it pumps. Valveless pumps can pump either way, and their net flow is highly sensitive to the pinching frequency (impedance pumping: Hickerson, Rinderknecht and Gharib 2005; chapter 4b's tracer pump is one).

THE THREE-ELEMENT HILL MODEL



A contractile element (CE) in series with an elastic element (SE), the pair in parallel with another elastic element (PE).

- CE: the active force of the actin–myosin crossbridges; fully extensible when inactive, able to shorten when activated. Use the combined law: $F_{CE} = a_F F_{\max} F_1(L_{CE}) F_2(V_{CE})$.
- PE: the passive connective tissue; resists stretch even when the CE is off.
- SE: the tendon and the filaments' own elasticity; stores energy.

Forces and lengths:

$$F = F_{PE} + F_{SE}, \quad F_{CE} = F_{SE}; \qquad L = L_{PE}, \quad L = L_{CE} + L_{SE};$$

with nonlinear springs, e.g. $F_{SE} = F_{CE} = k_{SE}(L - L_{CE})^3$ (zero resting length) and $F_{PE} = k_{PE}(L - L_R)^3$ (resting length L_R). "There are many other ways to model each elastic element, and these will depend upon the animal considered."

The 2015 lecture, slides 11–16. The series element and the CE carry the same force: given L, solve $F_{CE}(L_{CE}, \dot{L}_{CE}) = k_{SE}(L - L_{CE})^3$ for L_{CE} every step.

WHAT IB2D'S THREE-ELEMENT CODE COMPUTES (MATLAB)

`IBM_Driver.m`'s `read_Hill_3Muscle_Points` reads nine columns per muscle from `<name>.3_muscle` (lines 2057–2072), and `give_3_Element_Muscle_Force_Densities` uses them: `leader follower LFO SK a b Fmax kSpr alpha`. For each muscle, with $d = \mathbf{X}_f - \mathbf{X}_l$, $L_F = |d|$ and $v = |L_F - L_F^{\text{prev}}|/\Delta t$, the example's activation file returns F_m (the combined law), a switch `on_Parallel`, a coefficient `PE_Coeff` and the activation a_f, and the leader receives

$$\underbrace{F_m \frac{d}{L_F}}_{\text{CE}} \underbrace{- a_f \, k_{\text{Spr}} \, d \, + \, 1.0 \cdot \dot{\mathbf{X}}_l}_{\text{“SE”}} + \underbrace{\text{PE_Coeff} \cdot \tfrac{1}{2}(\alpha + 1) \, k_{\text{Spr}} \, (L_F - L_{FO})^{\alpha} \, \frac{d}{L_F}}_{\text{PE, if on_Parallel}},$$

the follower the opposite (lines 671–753; the "SE" term is coded as $a_f k_{\text{Spr}}((L_{FO} - L_F) - L_{FO})\, d/L_F$, with the comment "L_con (LF) + L_ser = L_tot = LFO").

- The CE and SE forces are added, not set equal; there is no separate L_{CE}, so it is not the series arrangement of the previous slide.
- The "SE" part is $-a_f k_{\text{Spr}}\, d$ on the leader: it pushes the two points *apart*, in proportion to their distance, while active.
- The damping term adds $1.0\times$ the leader's velocity to the leader's force: a force *along* the motion, the opposite sign of a damper, and it acts whether or not the muscle is active.

`matIB2d/IBM_Blackbox/please_Find_Lagrangian_Forces_On_Eulerian_grid.m`, read, not run (Octave cannot run matIB2d).

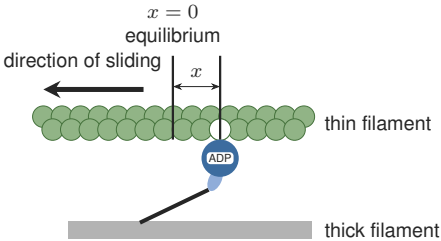
THE THREE-ELEMENT MODEL IN PRACTICE: WHICH EXAMPLES MATCH THE CODE

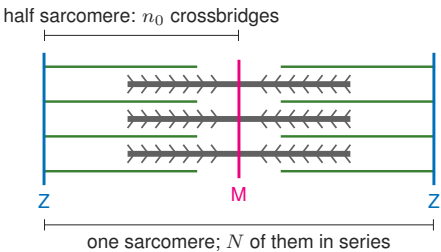
- MATLAB. The black box calls `[Fm,on_Parallel,PE_Coeff,af_Val] = give_3_Element_Muscle_Activation(v,LF,LFO,sk,a,b,Fmax, current_time,xPt,xLag,i)`: eleven inputs, four outputs. Of the three examples that use the model, only `Example_Scallop/Scallop_3Elem_Muscle` has a matching activation file (and the `.3_muscle` file). `Scallop_3Elem_Muscle_Almost_Swims` defines ten inputs and three outputs; `Example_HeartTube/3_Element_Muscle_Model` defines ten inputs and one output, and has no `HeartTube.3_muscle`. By their signatures, those two do not run with the 2024 black box.
- Python. The README (`pyIB2d/README.txt`) marks the model "UNTESTED!! THIS IS LIKELY BROKEN." It is: the reader takes seven columns from `.muscle` (no `kSpr`, `alpha`), the force routine has no SE or PE terms, and it calls a function it never imports. The heart tube with `FV_LT_muscle = 0` and `3_element_muscle = 1` stops at the first step:

```
  File ".../please_Find_Lagrangian_Forces_On_Eulerian_grid.py", line 465,
    in give_3_Element_Muscle_Force_Densities
    Fm = give_3_Element_Muscle_Activation(v,LF,LFO_Vec,SK_Vec,a_Vec,b_Vec,\
NameError: name 'give_3_Element_Muscle_Activation' is not defined. Did you mean: 'give_3_Element_Muscle_Force_Densities'?
```

For a three-element muscle in IB2d today: MATLAB and the scallop's files, read with the previous slide in mind; or write it as a user-defined force (slide 25), with the series element solved properly.

PART 3: CROSSBRIDGES



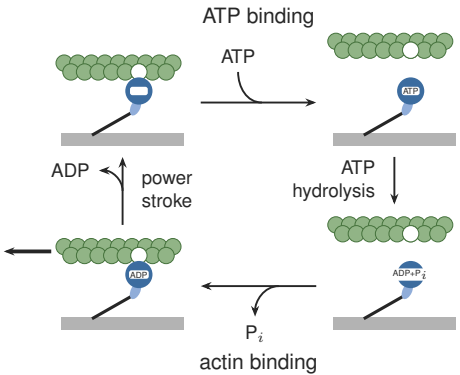


- Equilibrium configuration: where the crossbridge exerts no force on the thin filament; x is the distance from it.
- $p(x)$: the force on the thin filament when the crossbridge is displaced by x; $p(0) = 0$.
- n_0: the number of crossbridges in a half sarcomere; they act in parallel (forces add): all attached and displaced by x give $n_0\, p(x)$.
- N: the number of sarcomeres; the $2N$ half sarcomeres are in series (lengths add).
- $u(x)$: the population density of attached crossbridges. The fraction attached between x_1 and x_2 is $\int_{x_1}^{x_2} u\, dx$; the total fraction attached is $U = \int_{-\infty}^{\infty} u\, dx$; and the force on the ends of the muscle is

$$P = n_0 \int_{-\infty}^{\infty} p(x)\, u(x)\, dx.$$

The 2015 lecture, slides 19–22; the crossbridge (its slide 20) and the sarcomere (its slide 21) redrawn here.

ATTACH, SLIDE, DETACH



The 2015 lecture, slide 18 (redrawn): the ATPase–crossbridge cycle after Lymn and Taylor (1971).

- Attach: crossbridges attach at $x = A > 0$, at a rate proportional to the number available, $n_0(1 - U)$: rate $\alpha\, n_0(1 - U)$.
- Slide: attached crossbridges move with the filament, $dx/dt = -v$, v the shortening speed of the thin filament relative to the thick one; for the whole muscle $V = 2Nv$.
- Detach: at a rate β per attached crossbridge, independent of x: from (x_1, x_2) at rate $\beta n_0 \int_{x_1}^{x_2} u\, dx$; in total $\beta n_0 U$.

In steady shortening, crossbridges with $x_0 < x < A$ are replenished by attachment and lost by detachment and by sliding past x_0 (rate $v\, n_0\, u(x_0)$). Balance:

$$\alpha\,(1 - U) = \beta \int_{x_0}^{A} u(x)\, dx + v\, u(x_0).$$

THE STEADY STATE

The balance holds for every $x_0 < A$. Differentiate with respect to x_0:

$$0 = -\beta\, u(x_0) + v\,\frac{du}{dx_0} \quad\Longrightarrow\quad v\,\frac{du}{dx} = \beta u, \qquad u(x) = u(A)\,\exp\Big(\frac{\beta(x-A)}{v}\Big)\ \ (x < A),$$

and $u(x) = 0$ for $x > A$ (nothing attaches there). Two conditions fix $u(A)$ and U:

$$x_0 = A:\ \ \alpha(1-U) = v\,u(A); \qquad U = \int_{-\infty}^{A} u(A)\,e^{\beta(x-A)/v}\,dx = \frac{v\,u(A)}{\beta};$$

so

$$U = \frac{\alpha}{\alpha+\beta}, \qquad u(A) = \frac{\alpha\beta}{v\,(\alpha+\beta)}, \qquad u(x) = \frac{\alpha\beta}{v\,(\alpha+\beta)}\,\exp\Big(\frac{\beta(x-A)}{v}\Big).$$

The fraction attached does not depend on the speed; how far the attached bridges are dragged before they let go does: the faster the shortening, the more bridges sit at $x < 0$ pulling backwards.

The 2015 lecture, slides 28–32, which give no source for this form: Huxley's (1957) crossbridge model, simplified to attachment at one point and detachment at a constant rate.

HILL'S LAW FROM CROSSBRIDGES

With the nonlinear crossbridge spring $p(x) = p_1(e^{\gamma x} - 1)$ (a linear $p = kx$ gives a linear force–velocity curve),

$$P = \frac{\alpha\beta}{v(\alpha+\beta)} \int_{-\infty}^{A} n_0 p(x)\, e^{\beta(x-A)/v}\, dx$$

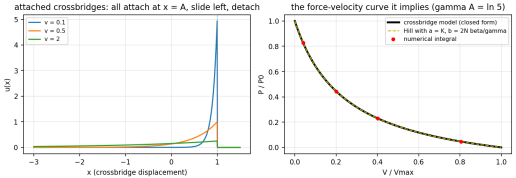
$$= K\, \frac{(e^{\gamma A} - 1) - \gamma v/\beta}{1 + \gamma v/\beta},$$

$K = \alpha n_0 p_1/(\alpha+\beta)$. Solve for $\gamma v/\beta$ and use $V = 2Nv$:

$$V = b\, \frac{P_0 - P}{P + a}, \quad a = K, \quad b = \frac{2N\beta}{\gamma}, \quad P_0 = K(e^{\gamma A} - 1),$$

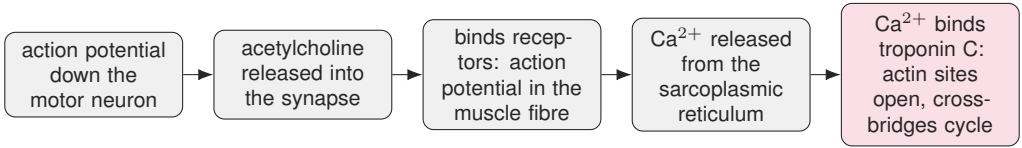
$$V_{\max} = \frac{2N\beta}{\gamma}\,(e^{\gamma A} - 1), \quad \frac{a}{P_0} = \frac{1}{e^{\gamma A} - 1}.$$

"Using this analysis, we have derived the empirical force–velocity curve from a model of crossbridge dynamics." Hill's $a/P_0 \approx 0.25$ means $\gamma A = \ln 5$.



`Muscle_Models/muscle_curves.py` with $\alpha = \beta = A = n_0 p_1 = N = 1$, $\gamma = \ln 5$: left, $u(x)$ at three speeds; right, the closed form (black), Hill's law with the derived a, b (yellow dashes) and the integral $n_0 \int p\, u\, dx$ computed numerically (red dots, four of the six speeds checked). The closed form and the numerical integral agree to $5 \cdot 10^{-8}$ of P_0 at six speeds.

PART 4: ACTIVATION, FROM NERVE TO CROSSBRIDGE



- If enough acetylcholine binds, an action potential is induced in the muscle fibre; it releases calcium ions from the sarcoplasmic reticulum into the cell.
- Contraction happens when calcium bound to troponin opens the myosin-binding sites on actin, so crossbridges can form and cycle, producing force.
- End of contraction: acetylcholinesterase in the synapse breaks acetylcholine down within milliseconds; with no acetylcholine on the receptors, calcium is taken back into the sarcoplasmic reticulum and the binding sites close.

The activation tutorial, slides 2–7 (its textbook figures are described here, not copied).

VARYING THE FORCE: SUMMATION, FIBRE TYPES, TETANUS

Nerve impulses are, for the most part, all alike; muscles still produce graded forces, by force summation:

- Multiple fibre summation: a weak signal recruits the small, more excitable motor units first; stronger signals add more and larger units (the largest up to about 50 times stronger than the smallest).
- Frequency summation: action potentials arriving faster; twitches add before the previous one has relaxed. At high enough frequency the muscle does not relax between stimuli: a tetanic (fused) contraction. During a maximal voluntary effort only about a third of a human muscle's fibres fire at once, which protects the tendon.

Fibre types. A muscle mixes fibre types in proportions that vary within and across species and with training. Slow-twitch fibres contract more slowly and fatigue late; fast-twitch fibres release calcium faster, split ATP faster and fatigue sooner. Chicken legs (walking, standing: dark meat) are mostly slow-twitch; breast and wings (short bursts of flight: white meat) mostly fast-twitch.

The activation tutorial, slides 8–14 (the summation text is quoted there from Wikipedia, the chicken example from a BBC Science page).

A CALCIUM MODEL OF ACTIVATION

Williams et al.'s model for lamprey muscle: the stimulus from the motor neurons is on or off; the action potential, acetylcholine and depolarisation are not modelled. Mass-action kinetics move calcium between the sarcoplasmic reticulum (bound: $[cs]$, free sites $[s]$), the cytosol ($[c]$) and the thin filaments (bound $[cf]$, free $[f]$): stimulus on, release at rate $k_1[cs]$; off, uptake at $k_2[c][s]$; binding to filaments $k_3[c][f]$, unbinding $k_4[cf][f]$. Conservation: $[cs]+[c]+[cf]=C_T$, $[cs]+[s]=S_T$, $[cf]+[f]=F_T$. Scale by the filament sites, $Ca=[c]/F_T$, $Caf=[cf]/F_T$, $C=C_T/F_T$, $S=S_T/F_T$ (so $Caf\le 1$).

Stimulus on:

$$\frac{dCa}{dt} = (k_4 Caf - k_3 Ca)(1-Caf) + k_1(C-Ca-Caf),$$

stimulus off:

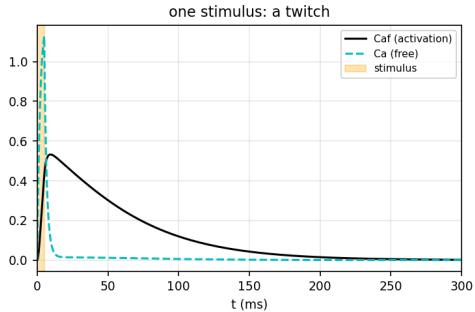
$$\frac{dCa}{dt} = (k_4 Caf - k_3 Ca)(1-Caf) + k_2\,Ca\,(C-S-Ca-Caf),$$

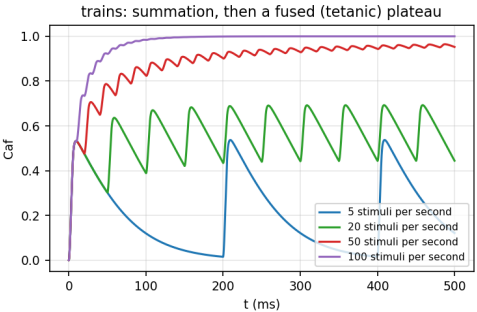
and always

$$\frac{dCaf}{dt} = -(k_4 Caf - k_3 Ca)(1-Caf).$$

Caf, the fraction of filament sites with calcium bound, is the activation: it multiplies the contractile force (slide 48). The rate constants absorb the factors of F_T from the scaling.

TWITCHES, SUMMATION AND TETANUS FROM THE CALCIUM MODEL





`Muscle_Models/calcium_activation.py` (fixed-step RK4, $\Delta t = 10^{-5}$ s), with illustrative constants, not fitted values: $k_1 = 300$, $k_2 = 100$, $k_3 = 150$, $k_4 = 30$ per second, $C = 2$, $S = 6$, stimuli of 5 ms.

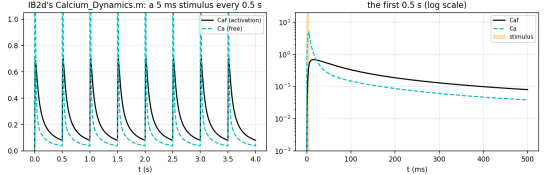
One stimulus: Caf peaks at 0.53 after 9 ms and halves at about 57 ms. Trains: at 5 per second the twitches stay separate (0.02–0.54 over the last 0.1 s), at 20 they sum (0.44–0.69), at 50 they fuse into a plateau (0.94–0.96), at 100 the filaments saturate ($Caf = 1.00$).

IB2D'S OWN CALCIUM MODEL

The MATLAB heart tubes with calcium (`Example_HeartTube/`: `Electromechanical_Pumping_w_Ca_Dynamics`, its `128x128_` version and `EM_Pump_w_Calcium_and_Muscles`; slide 49) solve this model in advance (`Calcium_Dynamics.m`, the same constants in every copy) and feed $0.5\,Caf(t)$ as the stimulus current of a FitzHugh–Nagumo cable between 25 and 30 % of its length:

$$\begin{aligned}\frac{dCa}{dt} &= k_4 Caf - k_3 Ca(1-Caf)\\ &\quad + k_2\,Ca\,(C - S - Ca - Caf)\\ &\quad \big[+\,k_1(C - Ca - Caf)\big],\\ \frac{dCaf}{dt} &= -k_4 Caf + k_3 Ca(1-Caf),\end{aligned}$$

the bracket only during a stimulus. Against slide 45: uptake never stops, and unbinding is $k_4\,Caf$, not $k_4\,Caf\,(1-Caf)$ (the file: "removed weird non-biological term"). Forward Euler with $\Delta t = 10^{-5}$ s; $k_1 = 9$, $k_2 = 50$, $k_3 = 40$, $k_4 = 19.4$ per second, $C = S = 200$; a 5 ms stimulus every 0.5 s for the run's 4 s.



`calcium_activation.py --ib2d`, the file line for line (left, the free calcium is clipped): each stimulus lifts the free calcium to 5.0; Caf peaks at 0.69 (0.70 from the second stimulus on, 2 % higher) 15 ms after the stimulus starts, halves by 96 ms and is still 0.08 when the next one comes: separate twitches. The `128x128_` copy still ends with its test plot and `pause()`, so in MATLAB that example stops at the plot until a key is pressed.

IB2d: `matIB2d/Examples/Example_HeartTube/*/Calcium_Dynamics.m` and `FitzHugh_Nagumo_1d.m` (N. A. Battista, 2016).

FROM CALCIUM TO FORCE: THE TWO-ELEMENT MODEL

Drop the parallel element; keep the contractile element in series with a linear elastic element of stiffness μ and resting length 0. With P the force, L and V the length and lengthening speed of the whole muscle (negative V is shortening), the contractile element has

$$l_c = L - P/\mu, \qquad v_c = V - \frac{dP/dt}{\mu}.$$

In some muscles the series elasticity resides largely in the activated filaments, so it depends on activation: $\mu(t) = \mu_0 + \mu_1\, Caf(t)$, and then

$$v_c = V - \frac{dP}{dt}\Big/\mu + \mu_1\, P\, \frac{dCaf}{dt}\Big/\mu^2.$$

The force of the contractile element:

$$P_c = P_0\, \lambda(l_c)\, \alpha(v_c)\, Caf, \qquad \lambda(l_c) = 1 + \lambda_2\, (l_c - l_{c0})^2, \qquad \alpha(v_c) = 1 + \begin{cases} \alpha_m v_c, & v_c < 0, \\ \alpha_p v_c, & v_c \geq 0, \end{cases}$$

l_{c0} the optimal length (λ has its maximum, 1, there if $\lambda_2 < 0$), and a linear force–velocity law that may differ between shortening and lengthening ($\alpha_m \neq \alpha_p$). In series, $P = P_c$: one ODE for P per element, driven by Caf.

The activation tutorial, slides 21–29 (Williams et al.). The sign of V is that of its slide 23; its slide 26 calls V the shortening velocity.

FROM THESE MODELS TO IB2D

in the models	in IB2d's `FV_LT_muscle`	to get it
activation a_F, or $Caf(t)$	`af_Val` from your `give_Muscle_Activation`	solve the calcium ODEs (slide 45) inside the activation file, one state per muscle, driven by your stimulus pattern
length–tension F_1, $\lambda(l_c)$	`F1 = exp(-((Q-1)/SK)**2)`	the Gaussian, or replace it in the same file
force–velocity F_2, $\alpha(v_c)$	`F2` with `v = |dLF/dt|`	the sign of the velocity is lost before your file sees it (slide 32); recompute it from `LF` and a stored previous value
series elasticity	none (`FV_LT`); added, not in series (`3_element`, MATLAB)	a user-defined force (slide 25) that keeps l_c per muscle
calcium and an excitable medium	MATLAB: FitzHugh–Nagumo heart tubes whose stimulus is $0.5\,Caf(t)$ from a calcium model (`Calcium_Dynamics.m`); the potential drives muscles only in the first example named here	`Example_HeartTube/EM_Pump_w_Calcium_and_Muscles`, `128x128_Electromechanical_Pumping_w_Ca_Dynamics`

Published uses of these ideas with the immersed boundary method: lamprey swimming with a kinetic, calcium-activated muscle model (Tytell et al. 2010, after McMillen, Williams and Holmes 2008, who modified Williams et al.'s calcium kinetics), the muscle models in IB2d (Battista, Baird and Miller 2015), in this tutorial, active pumping with custom spring forces in IBAMR (chapter 9b) and active tension in IBFE (chapter 20).

EXERCISES

1. Re-run `Gravity_Cellular_Race` with walls: a channel of target points around the three cylinders (as the MATLAB `Gravity_vs_Flow` race has, without its flow; the MATLAB `Gravity_Cellular_Race`, like the Python one, has none); repeat the momentum check of slide 14 and find the terminal speeds relative to the fluid.
2. Give the porous band springs of non-zero resting length (`L` = the initial spacing): does it still collapse? Plot its area against time for three permeabilities and fit the rate.
3. Change `User_Defined_Fiber_Model` into a cubic spring ($F = k(|d| - L)^3$) and compare the band with the linear one; then into a spring that only pulls (no force when $|d| < L$).
4. In `Concentration_Diffusion`, set $\mu = 0.01$ and stir the box with an external force; check whether the total amount is still conserved (slide 24: the upwind advection does not conserve it in a moving fluid), and measure the mixing time.
5. Give the heart tube's activation file a stored previous length, so that lengthening muscles get an eccentric branch (force above $F_{\max}$, which Hill's law, for shortening only, does not have; e.g. the α_p branch of slide 48) instead of the shortening one; compare the pinch and the flux.

EXERCISES (2)

6. Set the heart tube's `a` to $0.25\,F_{\max}$ (Hill's a/P_0) and run 20 periods; measure the mean flux at three sections, then vary the pulse speed (the `freq`) and find where the direction of pumping changes.
7. Replace the heart tube's square pulse by the calcium model: each muscle stimulated for 5 ms when the wave passes, its Caf integrated in the activation file; compare a 10 Hz train with a single wave.
8. Show from the slide 42 formulas that a linear crossbridge spring $p = kx$ gives a linear force–velocity curve; plot it against the exponential spring's.
9. With `calcium_activation.py`, find the stimulus frequency at which the twitches fuse (the minimum of Caf over a period reaches 90 % of its maximum) for three values of k_4; relate it to the twitch's half-decay time.
10. Write the two-element model of slide 48 for an isometric muscle ($V = 0$) and integrate $P(t)$ for a twitch and for a tetanus.

SUMMARY

- IB2d's models are forces between points (springs, beams, targets, masses, muscles, your own law), velocities added to points (porous, poroelastic), forces on the grid (external forcing, Boussinesq), and fields carried by the flow (concentration, tracers). Each has a file, a flag and an example; the Python examples run in seconds on a desktop PC (the two big ones in minutes, and hours to their shipped end times).
- Masses are per unit length and their weight reaches the fluid times Δs; in a periodic box nothing holds it up, so the whole fluid falls with them (momentum checked to 0.5 %, 1 % at the last dump). A concentration is conserved to 10^{-12} when the fluid is at rest; the upwind advection gains 0.36 % in the Rayleigh–Taylor run. A user-defined spring reproduces the built-in one exactly.
- The shipped porous bands collapse to a point just after their end time, whatever the step; the cause is the model (zero resting length plus slip), not the time step.
- Muscle force in IB2d is activation $\times$ length–tension $\times$ force–velocity along the line of two points. The code uses the speed without its sign and Hill's a in force units; the three-element model is not the textbook arrangement in MATLAB and does not run in Python.
- Hill's law follows from crossbridges that attach at one place, slide and detach at a constant rate, with an exponential crossbridge spring (checked numerically to $5 \cdot 10^{-8}$); calcium kinetics turn a stimulus train into the activation that sums twitches into tetanus.
- Next: chapter 4f, meshes from images with MeshmerizeMe.

REFERENCES AND CREDITS

- Sources merged (slide 3): N. A. Battista and L. A. Miller, “Fluid Structure Interaction (FSI) and the Immersed Boundary Method (IB)” (talk, 7 September 2016; pictures on slides 9, 12, 15, 16, 17, 19); L. A. Miller, “Quick introduction to muscle models” (15 September 2015; its drawings redrawn on slides 39, 40) and “Muscle Activation Tutorial: the biology and Ca^{2+} models”. Quotations are from these.
- IB2d: https://github.com/nickabattista/IB2d (clone of 2024-03-06): `EXAMPLE_LIST.txt`, `pyIB2d/Examples/*`, `matIB2d/Examples/*`, the force routines and the driver. N. A. Battista, A. J. Baird, L. A. Miller, “A mathematical model and MATLAB code for muscle–fluid–structure simulations”, Integr. Comp. Biol. 55 (2015) 901–911; N. A. Battista, W. C. Strickland, L. A. Miller, “IB2d: a Python and MATLAB implementation of the immersed boundary method”, Bioinspir. Biomim. 12 (2017) 036003; N. A. Battista, W. C. Strickland, A. Barrett, L. A. Miller, “IB2d Reloaded: a more powerful Python and MATLAB implementation of the immersed boundary method”, Math. Meth. Appl. Sci. 41 (2018) 8455–8480.
- Muscle: A. V. Hill, “The heat of shortening and the dynamic constants of muscle”, Proc. R. Soc. Lond. B 126 (1938) 136–195; A. F. Huxley, “Muscle structure and theories of contraction”, Prog. Biophys. Biophys. Chem. 7 (1957) 255–318; A. M. Gordon, A. F. Huxley, F. J. Julian, “The variation in isometric tension with sarcomere length in vertebrate muscle fibres”, J. Physiol. 184 (1966) 170–192; R. W. Lymn, E. W. Taylor, “Mechanism of adenosine triphosphate hydrolysis by actomyosin”, Biochemistry 10 (1971) 4617–4624; J. Keener, J. Sneyd, *Mathematical Physiology II: Systems Physiology*, 2nd ed., Springer (2009), chapter 15 (muscle; further reading); T. L. Williams, G. Bowtell, N. A. Curtin, “Predicting force generation by lamprey muscle during applied sinusoidal movement using a simple dynamic model”, J. Exp. Biol. 201 (1998) 869–875; E. D. Tytell, C.-Y. Hsu, T. L. Williams, A. H. Cohen, L. J. Fauci, “Interactions between internal forces, body stiffness, and fluid environment in a neuromechanical model of lamprey swimming”, PNAS 107 (2010) 19832–19837; T. McMillen, T. Williams, P. Holmes, “Nonlinear muscles, passive viscoelasticity and body taper conspire to create neuromechanical phase lags in anguilliform swimmers”, PLoS Comput. Biol. 4 (2008) e1000157; A. I. Hickerson, D. Rinderknecht, M. Gharib, “Experimental study of the behavior of a valveless impedance pump”, Exp. Fluids 38 (2005) 534–540; B. Alberts et al., *Molecular Biology of the Cell*, 4th ed. (the lecture’s figures; redrawn, not copied).
- Flow benchmarks: S. Turek, J. Hron, “Proposal for numerical benchmarking of fluid–structure interaction between an elastic object and laminar incompressible flow”, in *Fluid–Structure Interaction*, Lecture Notes in Computational Science and Engineering 53, Springer (2006) 371–385.
- This chapter’s runs and scripts (October 2026): `wsl/final/ch4e_*.sh`, `tools/ch4e_figures.py`, `examples/IB2d/Muscle_Models` (`muscle_curves.py`, `activation_wave.py`, `calcium_activation.py`, README), with `examples/analysis/read_vtk_legacy.py`.